

# Assignment 5

**Name:** Parth Koshti

**PRN:** 24070126093

**Batch:** AIML B1

**Title:** Banking Application: Design an inheritance-based banking system with account types and overridden methods.

## Code 1) Account.java

```
public class Account {

    protected int accountNumber;

    protected double balance;

    protected customer customer;

    private ArrayList<String> transactionHistory = new ArrayList<>();

    public Account(int accountNumber, double balance, customer customer) {

        this.accountNumber = accountNumber;

        this.balance = balance;

        this.customer = customer;

    }

    public void deposit(double amount) {

        balance += amount;

        transactionHistory.add("Deposited: " + amount + ", Balance: " + balance);

        System.out.println("Amount Deposited: " + amount);

        System.out.println("Balance: " + balance);

    }

    public void withdraw(double amount) throws InsufficientBalanceException {

        if (amount > balance)
```

```

        throw new InsufficientBalanceException("Insufficient Balance");

        balance -= amount;

        transactionHistory.add("Withdrawn: " + amount + ", Balance: " + balance);

        System.out.println("Amount Withdrawn: " + amount);

        System.out.println("Balance: " + balance);
    }

    public void displayAccount() {
        customer.displayCustomer();

        System.out.println("Account Number: " + accountNumber);

        System.out.println("Balance: " + balance);
    }

    public void printTransactionHistory() {
        System.out.println("--- Transaction History ---");

        for (String t : transactionHistory) {
            System.out.println(t);
        }
    }
}

```

## **Code 2) Customer.java**

```

public class customer {
    public String customerID;
    public String firstName;
    public String lastName;
    public String email;
    public String phoneNumber;
    public String address;
}

```

```

protected String aadharID;

protected String panID;

public customer(String customerID, String firstName, String lastName, String email, String
phoneNumber, String address, String aadharID, String panID) {

    this.customerID = customerID;

    this.firstName = firstName;

    this.lastName = lastName;

    this.email = email;

    this.phoneNumber = phoneNumber;

    this.address = address;

    this.aadharID = aadharID;

    this.panID = panID;

}

public void displayCustomer() {

    System.out.println("Customer ID: " + customerID);

    System.out.println("Name: " + firstName + " " + lastName);

    System.out.println("Email: " + email);

    System.out.println("Phone: " + phoneNumber);

    System.out.println("Address: " + address);

    System.out.println("Aadhar ID: " + aadharID);

    System.out.println("PAN ID: " + panID);

}

}

```

### **Code 3) CurrentAccount.java**

```

public class CurrentAccount extends Account {

    double minimumBalance = 500;

    public CurrentAccount(int accountNumber, double balance, customer customer) {

        super(accountNumber, balance, customer);

    }

    public void checkMinimumBalance() {

```

```

        if(balance < minimumBalance)

            System.out.println("Minimum balance not maintained");

        else

            System.out.println("Minimum balance maintained");

    }

    public void withdraw(double amount) throws InsufficientBalanceException {

        if(amount > balance)

            throw new InsufficientBalanceException("Insufficient Balance");

        balance -= amount;

        System.out.println("Current Account Withdrawal: " + amount);

    }

}

```

#### **Code 4) SavingAccount.java**

```

public class SavingsAccount extends Account {

    double interestRate = 0.067;

    public SavingsAccount(int accountNumber, double balance, customer customer) {

        super(accountNumber, balance, customer);

    }

    public void calculateInterest() {

        double interest = balance * interestRate;

        System.out.println("Interest Earned: " + interest);

    }

    public void withdraw(double amount) throws InsufficientBalanceException {

        if(balance - amount < 500)

            throw new InsufficientBalanceException("Minimum balance 500 required");

        balance -= amount;

        System.out.println("Savings Withdrawal: " + amount);

    }

}

```

### Code 5) Loan.java

```
public class Loan {  
  
    double loanAmount;  
  
    int months;  
  
    double emi;  
  
    public Loan(double loanAmount, int months) {  
  
        this.loanAmount = loanAmount;  
  
        this.months = months;  
  
        this.emi = loanAmount / months;  
  
    }  
  
    public void payEMI(Account acc) throws InsufficientBalanceException {  
  
        if(acc.balance < emi)  
  
            throw new InsufficientBalanceException("Not enough balance to pay EMI");  
  
        acc.balance -= emi;  
  
        System.out.println("EMI Paid: " + emi);  
  
        System.out.println("Remaining Balance: " + acc.balance);  
  
    }  
  
    public void displayLoan() {  
  
        System.out.println("Loan Amount: " + loanAmount);  
  
        System.out.println("Monthly EMI: " + emi);  
  
    }  
  
}
```

### Code 6) Main.java

```
import java.util.ArrayList;  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        try {  
  
            ArrayList<customer> customers = new ArrayList<>();  
  
            customer c1 = new customer("1`", "Parth", "Koshti", "kavishnag13@gmail.com",  
"9829334567", "Pune", "12345678910", "ABCDEFGHJK");
```

```

        customer c2 = new customer("2", "Kavish", "Nag", "kavish.nag@gmail.com", "9876534567",
"Pune", "98765432109", "XYZABCDEFGH");

        customer c3 = new customer("3", "krrish", "koshti", "krrish.koshti@gmail.com",
"9123345678", "Pune", "12341234123", "PQRSTUVWXYZ");

        customers.add(c1);

        customers.add(c2);

        customers.add(c3);

        ArrayList<Account> accounts = new ArrayList<>();

        SavingsAccount sa1 = new SavingsAccount(1001, 50000, c1);

        CurrentAccount ca1 = new CurrentAccount(2001, 30000, c1);

        accounts.add(sa1);

        accounts.add(ca1);

        SavingsAccount sa2 = new SavingsAccount(1002, 5000, c2);

        CurrentAccount ca2 = new CurrentAccount(2002, 150000, c2);

        accounts.add(sa2);

        accounts.add(ca2);

        SavingsAccount sa3 = new SavingsAccount(1003, 8000, c3);

        accounts.add(sa3);

        System.out.println("\n Performing Transactions \n");

        sa1.deposit(2000);

        sa1.withdraw(1000);

        sa1.calculateInterest();

        sa1.printTransactionHistory();

        System.out.println();

        Loan loan = new Loan(12000, 12);

        loan.displayLoan();

        loan.payEMI(sa1);

        System.out.println("\n Customer and Account summary:\n");

        displayInfo(customers, accounts);

    } catch (InsufficientBalanceException e) {

```

```

        System.out.println("Exception: " + e.getMessage());
    }
}

public static void displayInfo(ArrayList<customer> customers, ArrayList<Account> accounts)
{
    for(customer cust : customers) {
        System.out.println("---- CUSTOMER DETAILS ----");
        cust.displayCustomer();

        System.out.println("\n---- ACCOUNTS ----");

        double totalBalance = 0;
        int accountCount = 0;

        for(Account acc : accounts) {
            if(acc.customer.customerID.equals(cust.customerID)) {
                System.out.println("Account Number: " + acc.accountNumber);
                System.out.println("Account Type: " + acc.getClass().getSimpleName());
                System.out.println("Balance: Rs. " + acc.balance);
                acc.printTransactionHistory();
                totalBalance += acc.balance;
                accountCount++;
                System.out.println();
            }
        }
    }
}
}

```

### **Code 7) InsufficientBalanceException.java**

```

public class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String msg) {
        super(msg);}}

```

## Code 8) InsufficientAccountException.java

```
public class InvalidAccountException extends Exception {  
    public InvalidAccountException(String msg) {  
        super(msg);  
    }  
}
```

## Output:

```
Performing Transactions  
  
Amount Deposited: 2000.0  
Balance: 52000.0  
Savings Withdrawal: 1000.0  
Interest Earned: 3417.0  
--- Transaction History ---  
Deposited: 2000.0, Balance: 52000.0  
  
Loan Amount: 12000.0  
Monthly EMI: 1000.0  
EMI Paid: 1000.0  
Remaining Balance: 50000.0  
  
Customer and Account summary:  
  
---- CUSTOMER DETAILS ----  
Customer ID: 1  
Name: Parth Koshti  
Email: kavishnag13@gmail.com  
Phone: 9829334567  
Address: Pune  
Aadhar ID: 12345678910  
PAN ID: ABCDEFGHIJK  
  
---- ACCOUNTS ----  
Account Number: 1001  
Account Type: SavingsAccount  
Balance: Rs. 50000.0  
--- Transaction History ---  
Deposited: 2000.0, Balance: 52000.0  
  
Account Number: 2001  
Account Type: CurrentAccount  
Balance: Rs. 30000.0
```

```
--- Transaction History ---  
  
---- CUSTOMER DETAILS ----  
Customer ID: 2  
Name: Kavish Nag  
Email: kavish.nag@gmail.com  
Phone: 9876534567  
Address: Pune  
Aadhar ID: 98765432109  
PAN ID: XYZABCDEFGH  
  
---- ACCOUNTS ----  
Account Number: 1002  
Account Type: SavingsAccount  
Balance: Rs. 5000.0  
--- Transaction History ---  
  
Account Number: 2002  
Account Type: CurrentAccount  
Balance: Rs. 150000.0  
--- Transaction History ---  
  
---- CUSTOMER DETAILS ----  
Customer ID: 3  
Name: krrish koshti  
Email: krrish.koshti@gmail.com  
Phone: 9123345678  
Address: Pune  
Aadhar ID: 12341234123  
PAN ID: PQRSTUWXYZ  
  
---- ACCOUNTS ----  
Account Number: 1003  
Account Type: SavingsAccount  
Balance: Rs. 8000.0  
--- Transaction History ---
```

# Exceptions:

## 1) Insufficient Balance Exception

```
Performing Transactions

Amount Deposited: 200.0
Balance: 700.0
Savings Withdrawal: 100.0
Interest Earned: 40.2
--- Transaction History ---
Deposited: 200.0, Balance: 700.0

Loan Amount: 12000.0
Monthly EMI: 1000.0
Exception: Not enough balance to pay EMI
(base) PS D:\STUDY\Java Lab\BankingWithArrayLists\k> █
```

## 2) Invalid Account Exception

```
Exception: Account number 9999 not found.

Performing Transactions

Amount Deposited: 200.0
Balance: 700.0
Savings Withdrawal: 100.0
Interest Earned: 40.2
--- Transaction History ---
Deposited: 200.0, Balance: 700.0

Loan Amount: 12000.0
Monthly EMI: 1000.0
Exception: Not enough balance to pay EMI
(base) PS D:\STUDY\Java Lab\BankingWithArrayLists\k> █
```